

Online Judge Platform

Prepared By: Kuswanth Tumma

Database Design & Modeling Specification

Online Judge Platform — MongoDB Document Architecture

| Document Metadata | Details |

| :--- | :--- |

| **Project Title** | Online Judge Platform |

| **Database System** | MongoDB v6.0+ Enterprise / Atlas |

| **Document Version** | 1.0.0-RELEASE |

| **Status** | Approved for Production |

| **Author** | Kuswanth Tumma |

| **Date** | July 2026 |

1. Data Store Selection Rationale & Trade-off Analysis

Choosing an appropriate database engine is critical for an Online Judge. The decision between a Relational Store (PostgreSQL) and a Document Store (MongoDB) was evaluated based on workload characteristics:

DATABASE ARCHITECTURAL COMPARISON MATRIX		
Characteristic	PostgreSQL (Relational)	MongoDB (Document)
Testcase Inputs/Outputs	Normalization overhead	Embedded / Document
Problem Schema Flexibility	DDL migration cost	Schema-less / Dynamic
Write Throughput (Submits)	High lock contention	High append throughput
Leaderboard Aggregation	SQL GROUP BY heavy	Aggregation Pipeline

Decision: MongoDB was selected due to its high write throughput for rapid code submission logs, dynamic schema adaptation for evolving problem statement metadata, and native JSON array embedding for testcase definitions.

2. Entity Relationship (ER) Diagram

The logical relationships between platform domain entities are illustrated in the Mermaid ER diagram below:

```
erDiagram
    USERS ||--o{ SUBMISSIONS : "submits"
    USERS ||--o{ PROBLEMS : "creates (Admin)"
    PROBLEMS ||--o{ TESTCASES : "contains"
    PROBLEMS ||--o{ SUBMISSIONS : "evaluates"
    USERS ||--o{ LEADERBOARDS : "ranks"

    USERS {
        ObjectId _id PK
        string name
        string email UK
        string passwordHash
        string role
        number problemsSolved
        number totalSubmissions
        date createdAt
    }

    PROBLEMS {
        ObjectId _id PK
        string title
        string slug UK
        string statement
        string difficulty
        object constraints
        string sampleInput
        string sampleOutput
        ObjectId createdBy FK
        date createdAt
    }

    TESTCASES {
        ObjectId _id PK
        ObjectId problemId FK
        string input
        string expectedOutput
        boolean isHidden
        number order
    }

    SUBMISSIONS {
        ObjectId _id PK
        ObjectId userId FK
        ObjectId problemId FK
        string language
        string sourceCode
        string verdict
        number executionTimeMs
        number memoryUsedKb
        string errorMessage
```

```
    date submittedAt
  }

  LEADERBOARDS {
    ObjectId _id PK
    ObjectId userId FK UK
    number rank
    number problemsSolved
    number totalSubmissions
    number accuracy
    date updatedAt
  }
}
```

3. Detailed MongoDB Collections Specification

3.1 **users** Collection Schema

```
{
  "_id": { "$type": "objectId" },
  "name": { "$type": "string", "maxLength": 100 },
  "email": { "$type": "string", "pattern": "^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.\.[a-zA-Z]{2,}$" },
  "passwordHash": { "$type": "string" },
  "role": { "$enum": ["User", "Admin"] },
  "problemsSolved": { "$type": "int", "minimum": 0 },
  "totalSubmissions": { "$type": "int", "minimum": 0 },
  "createdAt": { "$type": "date" },
  "updatedAt": { "$type": "date" }
}
```

3.2 **problems** Collection Schema

```
{
  "_id": { "$type": "objectId" },
  "title": { "$type": "string", "maxLength": 150 },
  "slug": { "$type": "string" },
  "statement": { "$type": "string" },
  "difficulty": { "$enum": ["Easy", "Medium", "Hard"] },
  "constraints": {
    "timeLimitMs": { "$type": "int", "default": 1000 },
    "memoryLimitMb": { "$type": "int", "default": 256 }
  },
  "sampleInput": { "$type": "string" },
  "sampleOutput": { "$type": "string" },
  "createdBy": { "$type": "objectId" },
  "createdAt": { "$type": "date" },
  "updatedAt": { "$type": "date" }
}
```

3.3 testcases Collection Schema

```
{
  "_id": { "$type": "objectId" },
  "problemId": { "$type": "objectId" },
  "input": { "$type": "string" },
  "expectedOutput": { "$type": "string" },
  "isHidden": { "$type": "bool", "default": true },
  "order": { "$type": "int" }
}
```

3.4 submissions Collection Schema

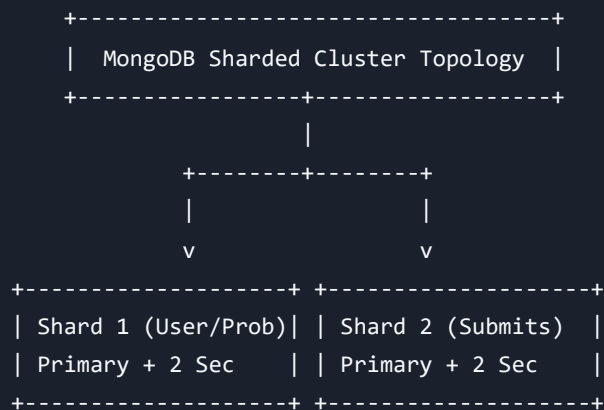
```
{
  "_id": { "$type": "objectId" },
  "userId": { "$type": "objectId" },
  "problemId": { "$type": "objectId" },
  "language": { "$enum": ["cpp", "python", "java"] },
  "sourceCode": { "$type": "string", "maxLength": 65536 },
  "verdict": { "$enum": ["Pending", "Processing", "Accepted", "Wrong Answer", "Time Limit Exceeded", "Memory Limit Exceeded", "Runtime Error", "Compilation Error"] },
  "executionTimeMs": { "$type": "int" },
  "memoryUsedKb": { "$type": "int" },
  "errorMessage": { "$type": "string" },
  "submittedAt": { "$type": "date" }
}
```

4. Indexing & Query Execution Strategy

To maintain sub-150ms query response times under high read/write volume, the collection indexes defined below are created:

Collection	Index Name	Key Fields	Type / Option	Purpose
users	idx_users_email	{ email: 1 }	Unique	Instant user auth lookup.
problems	idx_problems_slug	{ slug: 1 }	Unique	Fast problem page loading by URL slug.
problems	idx_problems_diff	{ difficulty: 1 }	Standard	Filtering problem list by difficulty.
testcases	idx_testcases_pid	{ problemId: 1, order: 1 }	Compound	Fast retrieval of evaluation testcases.
submissions	idx_sub_user_hist	{ userId: 1, submittedAt: -1 }	Compound	User submission history timeline queries.
submissions	idx_sub_verdict	{ problemId: 1, verdict: 1 }	Compound	Problem statistics & acceptance rate calculation.
leaderboards	idx_leaderboard	{ problemsSolved: -1, accuracy: -1 }	Compound	Fast global leaderboard ordering.

5. Database Replication, Sharding & Backup Policies



1. **Replication Strategy:** Primary-Secondary-Secondary replica set topology across separate Cloud Availability Zones (AZs) guarantees 99.999% data durability.

2. **Sharding Strategy:** The `submissions` collection is sharded using hashed sharding on `userId` (`{ userId: "hashed" }`), distributing high write volumes across multiple database nodes.

3. **Backup Policy:** Automated daily continuous snapshot backups with point-in-time recovery (PITR) retention for 30 days.